

当前文件: backend/main.py

```
// ===== File: backend/main.py =====
1  from __future__ import annotations
2
3  import json
4  import os
5  import re
6  import sqlite3
7  import uuid
8  from datetime import datetime
9  from functools import lru_cache
10 from io import BytesIO
11 from pathlib import Path
12 from typing import Any
13
14 from fastapi import FastAPI, File, UploadFile
15 from fastapi.middleware.cors import CORSMiddleware
16 from fastapi.responses import StreamingResponse
17 from pydantic import BaseModel
18
19 BASE_DIR = Path(__file__).resolve().parent
20 DATA_DIR = BASE_DIR / "data"
21 UPLOAD_DIR = DATA_DIR / "uploads"
22 DB_PATH = DATA_DIR / "questionnaire.db"
23 PADDLE_OCR_DIR = DATA_DIR / "paddleocr"
24
25 DATA_DIR.mkdir(exist_ok=True)
26 UPLOAD_DIR.mkdir(exist_ok=True)
27 PADDLE_OCR_DIR.mkdir(exist_ok=True)
28 os.environ.setdefault("PADDLE_OCR_BASE_DIR", str(PADDLE_OCR_DIR))
29 os.environ.setdefault("FLAGS_use_mkldnn", "0")
30 os.environ.setdefault("FLAGS_use_onednn", "0")
31
32 try:
33     import cv2
34     import numpy as np
35 except ImportError: # pragma: no cover
36     cv2 = None
37     np = None
38
39 try:
40     import easyocr
41 except Exception: # pragma: no cover
42     easyocr = None
43
44 try:
45     import paddle
46     from paddleocr import PaddleOCR
47     try:
48         paddle.set_flags({"FLAGS_use_mkldnn": False})
49     except Exception:
```

当前文件: backend/main.py

```
50         pass
51     try:
52         paddle.set_flags({"FLAGS_use_onednn": False})
53     except Exception:
54         pass
55 except Exception: # pragma: no cover
56     paddle = None
57     PaddleOCR = None
58
59 try:
60     from PIL import Image, ImageStat
61 except ImportError: # pragma: no cover
62     Image = None
63     ImageStat = None
64
65 try:
66     from openpyxl import Workbook
67 except ImportError: # pragma: no cover
68     Workbook = None
69
70
71 app = FastAPI(title="问卷图像识别与数据导出系统 V1.0")
72 app.add_middleware(
73     CORSMiddleware,
74     allow_origins=["http://127.0.0.1:5173", "http://localhost:5173"],
75     allow_credentials=True,
76     allow_methods=["*"],
77     allow_headers=["*"],
78 )
79
80
81 class ExportPayload(BaseModel):
82     taskId: str
83     rows: list[dict[str, Any]]
84
85
86 def connect() -> sqlite3.Connection:
87     connection = sqlite3.connect(DB_PATH)
88     connection.row_factory = sqlite3.Row
89     return connection
90
91
92 def init_db() -> None:
93     with connect() as db:
94         db.execute(
95             """
96             CREATE TABLE IF NOT EXISTS tasks (
97                 id TEXT PRIMARY KEY,
98                 name TEXT NOT NULL,
99                 created_at TEXT NOT NULL,
```

当前文件: backend/main.py

```
100         file_count INTEGER NOT NULL,
101         status TEXT NOT NULL
102     )
103     """
104 )
105 db.execute(
106     """
107     CREATE TABLE IF NOT EXISTS results (
108         id INTEGER PRIMARY KEY AUTOINCREMENT,
109         task_id TEXT NOT NULL,
110         question_no INTEGER NOT NULL,
111         question TEXT NOT NULL,
112         answer TEXT NOT NULL,
113         confidence INTEGER NOT NULL,
114         status TEXT NOT NULL,
115         raw_json TEXT NOT NULL,
116         FOREIGN KEY(task_id) REFERENCES tasks(id)
117     )
118     """
119 )
120
121
122 @lru_cache(maxsize=1)
123 def get_easyocr_reader() -> Any:
124     if easyocr is None:
125         raise RuntimeError("EasyOCR 未安装")
126     return easyocr.Reader(["ch_sim", "en"], gpu=False)
127
128
129 @lru_cache(maxsize=1)
130 def get_paddleocr_reader() -> Any:
131     if PaddleOCR is None:
132         raise RuntimeError("PaddleOCR 未安装")
133     try:
134         return PaddleOCR(
135             lang="ch",
136             ocr_version="PP-OCRv3",
137             ir_optim=False,
138             use_doc_orientation_classify=False,
139             use_doc_unwarping=False,
140             use_textline_orientation=False,
141         )
142     except Exception:
143         return PaddleOCR(use_angle_cls=True, lang="ch", ocr_version="PP-OCRv3", ir_optim=False, show_log=False)
144
145
146 def bbox_center(box: list[list[float]]) -> tuple[float, float]:
147     xs = [point[0] for point in box]
148     ys = [point[1] for point in box]
149     return sum(xs) / len(xs), sum(ys) / len(ys)
```

当前文件: backend/main.py

```
150
151
152 def bbox_rect(box: list[list[float]]) -> tuple[float, float, float, float]:
153     xs = [point[0] for point in box]
154     ys = [point[1] for point in box]
155     return min(xs), min(ys), max(xs), max(ys)
156
157
158 def run_easyocr(path: Path) -> dict[str, Any]:
159     reader = get_easyocr_reader()
160     raw_items = reader.readtext(str(path), detail=1, paragraph=False)
161     items = []
162     for box, text, confidence in raw_items:
163         clean_text = re.sub(r"\s+", " ", text).strip()
164         if not clean_text:
165             continue
166         x, y = bbox_center(box)
167         x1, y1, x2, y2 = bbox_rect(box)
168         items.append(
169             {
170                 "text": clean_text,
171                 "confidence": round(float(confidence) * 100, 1),
172                 "box": box,
173                 "x": x,
174                 "y": y,
175                 "rect": [x1, y1, x2, y2],
176             }
177         )
178     items.sort(key=lambda item: (item["y"], item["x"]))
179     return {
180         "items": items,
181         "text": "\n".join(item["text"] for item in items),
182         "avg_confidence": round(sum(item["confidence"] for item in items) / max(len(items), 1), 1),
183         "engine": "easyocr",
184     }
185
186
187 def normalize_paddle_box(poly: Any) -> list[list[float]]:
188     points = []
189     for point in poly:
190         points.append([float(point[0]), float(point[1])])
191     return points
192
193
194 def run_paddleocr(path: Path) -> dict[str, Any]:
195     reader = get_paddleocr_reader()
196     items = []
197
198     if hasattr(reader, "predict"):
199         prediction = reader.predict(str(path))
```

当前文件: backend/main.py

```

200         for page in prediction:
201             data = getattr(page, "json", None)
202             if callable(data):
203                 data = data()
204             if not isinstance(data, dict):
205                 data = page if isinstance(page, dict) else {}
206             result = data.get("res", data)
207             texts = result.get("rec_texts") or result.get("texts") or []
208             scores = result.get("rec_scores") or result.get("scores") or []
209             polys = result.get("rec_polys") or result.get("dt_polys") or result.get("polys") or []
210             for index, text in enumerate(texts):
211                 clean_text = re.sub(r"\s+", " ", str(text)).strip()
212                 if not clean_text:
213                     continue
214                 score = float(scores[index]) if index < len(scores) else 0.75
215                 box = normalize_paddle_box(polys[index]) if index < len(polys) else [[0, float(index * 32)], [1,
float(index * 32)], [1, float(index * 32 + 20)], [0, float(index * 32 + 20)]]
216                 x, y = bbox_center(box)
217                 x1, y1, x2, y2 = bbox_rect(box)
218                 items.append({"text": clean_text, "confidence": round(score * 100, 1), "box": box, "x": x, "y":
y, "rect": [x1, y1, x2, y2]})
219             else:
220                 raw_pages = reader.ocr(str(path), cls=True)
221                 for page in raw_pages or []:
222                     for line in page or []:
223                         if not line or len(line) < 2:
224                             continue
225                         box = normalize_paddle_box(line[0])
226                         text_info = line[1]
227                         text = text_info[0] if len(text_info) > 0 else ""
228                         score = float(text_info[1]) if len(text_info) > 1 else 0.75
229                         clean_text = re.sub(r"\s+", " ", str(text)).strip()
230                         if not clean_text:
231                             continue
232                         x, y = bbox_center(box)
233                         x1, y1, x2, y2 = bbox_rect(box)
234                         items.append({"text": clean_text, "confidence": round(score * 100, 1), "box": box, "x": x, "y":
y, "rect": [x1, y1, x2, y2]})
235
236             items.sort(key=lambda item: (item["y"], item["x"]))
237             return {
238                 "items": items,
239                 "text": "\n".join(item["text"] for item in items),
240                 "avg_confidence": round(sum(item["confidence"] for item in items) / max(len(items), 1), 1),
241                 "engine": "paddleocr",
242             }
243
244
245 def run_best_ocr(path: Path) -> dict[str, Any]:
246     errors = []

```

当前文件: backend/main.py

```
247     if PaddleOCR is not None:
248         try:
249             return run_paddleocr(path)
250         except Exception as exc: # noqa: BLE001 - OCR model/download/runtime failures should fall through.
251             errors.append(f"PaddleOCR失败: {exc}")
252     if easyocr is not None:
253         try:
254             result = run_easyocr(path)
255             if errors:
256                 result["fallbackReason"] = "; ".join(errors)
257             return result
258         except Exception as exc: # noqa: BLE001
259             errors.append(f"EasyOCR失败: {exc}")
260     raise RuntimeError("; ".join(errors) or "未安装可用OCR引擎")
261
262
263 def detect_checkboxes(path: Path) -> list[dict[str, Any]]:
264     if cv2 is None or np is None:
265         return []
266
267     image = cv2.imdecode(np.fromfile(str(path), dtype=np.uint8), cv2.IMREAD_GRAYSCALE)
268     if image is None:
269         return []
270
271     max_width = 1600
272     scale = 1.0
273     if image.shape[1] > max_width:
274         scale = max_width / image.shape[1]
275     image = cv2.resize(image, None, fx=scale, fy=scale, interpolation=cv2.INTER_AREA)
276
277     blurred = cv2.GaussianBlur(image, (3, 3), 0)
278     binary = cv2.adaptiveThreshold(
279         blurred,
280         255,
281         cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
282         cv2.THRESH_BINARY_INV,
283         31,
284         8,
285     )
286     contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
287     boxes: list[dict[str, Any]] = []
288     for contour in contours:
289         x, y, w, h = cv2.boundingRect(contour)
290         area = w * h
291         if area < 180 or area > 3600:
292             continue
293         aspect = w / max(h, 1)
294         if not 0.65 <= aspect <= 1.45:
295             continue
296         if w < 14 or h < 14 or w > 60 or h > 60:
```

当前文件: backend/main.py

```
297         continue
298
299     roi = binary[y : y + h, x : x + w]
300     fill_ratio = float(cv2.countNonZero(roi)) / max(area, 1)
301     if not 0.08 <= fill_ratio <= 0.75:
302         continue
303
304     perimeter = cv2.arcLength(contour, True)
305     approx = cv2.approxPolyDP(contour, 0.04 * perimeter, True)
306     contour_area = cv2.contourArea(contour)
307     extent = contour_area / max(area, 1)
308     circularity = 4 * np.pi * contour_area / max(perimeter * perimeter, 1)
309     looks_square = 4 <= len(approx) <= 6 and extent >= 0.48
310     looks_circle = len(approx) > 6 and 0.45 <= circularity <= 1.25
311     if not (looks_square or looks_circle):
312         continue
313     shape = "圆形" if len(approx) > 6 else "方形"
314     checked = fill_ratio > 0.28
315     boxes.append(
316         {
317             "x": round(x / scale),
318             "y": round(y / scale),
319             "w": round(w / scale),
320             "h": round(h / scale),
321             "shape": shape,
322             "checked": checked,
323             "fillRatio": round(fill_ratio, 3),
324         }
325     )
326
327     boxes.sort(key=lambda item: (item["y"], item["x"]))
328     filtered: list[dict[str, Any]] = []
329     for box in boxes:
330         is_duplicate = any(
331             abs(box["x"] - old["x"]) < 8 and abs(box["y"] - old["y"]) < 8 for old in filtered
332         )
333         if not is_duplicate:
334             filtered.append(box)
335     return filtered[:80]
336
337
338 def analyze_image(path: Path) -> dict[str, Any]:
339     if Image is None or ImageStat is None:
340         return {
341             "width": 0,
342             "height": 0,
343             "dark_ratio": 0,
344             "checkbox_candidates": 0,
345             "note": "未安装 Pillow, 无法读取图片。",
346         }
```

当前文件: backend/main.py

```
347
348     with Image.open(path) as image:
349         width, height = image.size
350         gray = image.convert("L")
351         gray.thumbnail((1200, 1200))
352         stat = ImageStat.Stat(gray)
353         mean_gray = stat.mean[0]
354         total_pixels = max(gray.size[0] * gray.size[1], 1)
355         dark_ratio = sum(1 for value in gray.getdata() if value < 120) / total_pixels
356         confidence = max(55, min(96, int(100 - abs(mean_gray - 210) / 3)))
357         return {
358             "width": width,
359             "height": height,
360             "dark_ratio": round(dark_ratio * 100, 2),
361             "confidence": confidence,
362         }
363
364
365 def nearest_option_text(box: dict[str, Any], ocr_items: list[dict[str, Any]]) -> str:
366     candidates = []
367     box_y = box["y"] + box["h"] / 2
368     box_x = box["x"] + box["w"]
369     for item in ocr_items:
370         item_y = item["y"]
371         item_x = item["x"]
372         if item_x < box_x:
373             continue
374         y_distance = abs(item_y - box_y)
375         if y_distance > max(28, box["h"] * 1.2):
376             continue
377         candidates.append((y_distance, item_x - box_x, item["text"]))
378     if not candidates:
379         return "未匹配到选项文字"
380     candidates.sort(key=lambda value: (value[0], value[1]))
381     return candidates[0][2]
382
383
384 def build_ocr_rows(task_id: str, file_results: list[dict[str, Any]]) -> list[dict[str, Any]]:
385     rows: list[dict[str, Any]] = []
386     for file_index, result in enumerate(file_results, start=1):
387         ocr = result.get("ocr", {})
388         boxes = result.get("checkboxes", [])
389         checked_boxes = [box for box in boxes if box.get("checked")]
390         ocr_items = ocr.get("items", [])
391         text_lines = [item["text"] for item in ocr_items]
392
393         if checked_boxes:
394             for box_index, box in enumerate(checked_boxes, start=1):
395                 option_text = nearest_option_text(box, ocr_items)
396                 rows.append(
```


当前文件: backend/main.py

```
397         {
398             "id": len(rows) + 1,
399             "taskId": task_id,
400             "question": f"图片 {file_index} 勾选项 {box_index}",
401             "answer": option_text,
402             "confidence": min(96, max(60, int(ocr.get("avg_confidence") or 75))),
403             "status": "待确认",
404             "type": box.get("shape", "选项框"),
405             "options": [
406                 f"位置: x={box['x']}, y={box['y']}, w={box['w']}, h={box['h']}",
407                 f"填充比例: {box['fillRatio']}",
408                 f"OCR文字总数: {len(text_lines)}",
409             ],
410         }
411     )
412
413     question_candidates = [
414         text for text in text_lines if re.search(r"^\d+[\.\.\,][? ?]|姓名|性别|年龄|满意|建议|是否)", text)
415     ]
416     if not question_candidates and text_lines:
417         question_candidates = text_lines[:8]
418
419     for text in question_candidates[:12]:
420         rows.append(
421             {
422                 "id": len(rows) + 1,
423                 "taskId": task_id,
424                 "question": text,
425                 "answer": "请人工确认",
426                 "confidence": min(96, max(55, int(ocr.get("avg_confidence") or 70))),
427                 "status": "待确认",
428                 "type": "OCR文本",
429                 "options": text_lines[:10],
430             }
431         )
432
433     if not rows:
434         analysis = result.get("analysis", {})
435         rows.append(
436             {
437                 "id": 1,
438                 "taskId": task_id,
439                 "question": f"图片 {file_index} 未识别到清晰文字",
440                 "answer": "请更换更清晰的扫描件或手动录入",
441                 "confidence": int(analysis.get("confidence") or 55),
442                 "status": "需复核",
443                 "type": "OCR异常",
444                 "options": [
445                     f"尺寸: {analysis.get('width')} x {analysis.get('height')}",
446                     f"深色像素占比: {analysis.get('dark_ratio')}%",
```

当前文件: backend/main.py

```
447         ],
448     }
449 )
450
451 rows.append(
452     {
453         "id": len(rows) + 1,
454         "taskId": task_id,
455         "question": "整批问卷识别摘要",
456         "answer": f"共识别 {sum(len(item.get('ocr', {})).get('items', [])) for item in file_results)} 段文字，检测
{sum(len(item.get('checkboxes', [])) for item in file_results)} 个候选选项框",
457         "confidence": 88,
458         "status": "待确认",
459         "type": "批量摘要",
460         "options": ["请在右侧人工校正题目文本、答案和备注。"],
461     }
462 )
463 return rows
464
465
466 def build_lightweight_rows(task_id: str, file_results: list[dict[str, Any]], error: str = "") -> list[dict[str,
Any]]:
467     rows: list[dict[str, Any]] = []
468     for index, result in enumerate(file_results, start=1):
469         analysis = result.get("analysis", {})
470         boxes = result.get("checkboxes", [])
471         rows.append(
472             {
473                 "id": len(rows) + 1,
474                 "taskId": task_id,
475                 "question": f"图片 {index} 基础识别结果",
476                 "answer": f"检测到 {len(boxes)} 个候选选项框",
477                 "confidence": int(analysis.get("confidence") or 70),
478                 "status": "待确认" if boxes else "需复核",
479                 "type": "轻量图像分析",
480                 "options": [
481                     f"尺寸: {analysis.get('width')} x {analysis.get('height')}",
482                     f"深色像素占比: {analysis.get('dark_ratio')}%",
483                     error or "OCR模型未启用，已完成基础图像分析。",
484                 ],
485             }
486         )
487     return rows
488
489
490 @app.on_event("startup")
491 def startup() -> None:
492     init_db()
493
494
```

当前文件: backend/main.py

```
495 @app.get("/api/health")
496 def health() -> dict[str, Any]:
497     return {
498         "status": "ok",
499         "name": "问卷图像识别与数据导出系统 V1.0",
500         "paddleocrInstalled": PaddleOCR is not None,
501         "paddleInstalled": paddle is not None,
502         "easyocrInstalled": easyocr is not None,
503         "opencvInstalled": cv2 is not None,
504     }
505
506
507 @app.post("/api/tasks")
508 async def create_task(files: list[UploadFile] = File(...)) -> dict[str, Any]:
509     task_id = f"TASK-{datetime.now():%Y%m%d}-{uuid.uuid4().hex[:6].upper()}"
510     task_name = Path(files[0].filename or "问卷图片").stem
511     saved_files = []
512     file_results = []
513     mode = "ocr"
514     mode_error = ""
515
516     for index, upload in enumerate(files):
517         suffix = Path(upload.filename or "").suffix.lower() or ".jpg"
518         filename = f"{task_id}_{index + 1}{suffix}"
519         target = UPLOAD_DIR / filename
520         content = await upload.read()
521         target.write_bytes(content)
522
523         analysis = analyze_image(target)
524         checkboxes = detect_checkboxes(target)
525         result: dict[str, Any] = {"analysis": analysis, "checkboxes": checkboxes}
526         try:
527             result["ocr"] = run_best_ocr(target)
528         except Exception as exc: # noqa: BLE001 - OCR engines fail for environment/model reasons.
529             mode = "lightweight"
530             mode_error = f"OCR模型暂不可用: {exc}"
531             result["ocrError"] = mode_error
532
533         file_results.append(result)
534         saved_files.append(
535             {
536                 "id": f"{task_id}-{index + 1}",
537                 "name": upload.filename,
538                 "size": f"{len(content) / 1024 / 1024:.2f} MB",
539                 "status": f"{result.get('ocr', {}).get('engine', 'OCR')}识别完成" if mode == "ocr" else "轻量识别完成",
540                 "progress": 100,
541                 "path": str(target),
542                 "analysis": analysis,
543                 "checkboxes": checkboxes,
544                 "ocrText": result.get("ocr", {}).get("text", ""),
```

当前文件: backend/main.py

```
545         "ocrError": result.get("ocrError", ""),
546     }
547 )
548
549 results = build_ocr_rows(task_id, file_results) if mode == "ocr" else build_lightweight_rows(task_id,
file_results, mode_error)
550 now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
551 status = "OCR识别完成" if mode == "ocr" else "轻量识别完成"
552 with connect() as db:
553     db.execute(
554         "INSERT INTO tasks (id, name, created_at, file_count, status) VALUES (?, ?, ?, ?, ?)",
555         (task_id, task_name, now, len(saved_files), status),
556     )
557     db.executemany(
558         """
559         INSERT INTO results (task_id, question_no, question, answer, confidence, status, raw_json)
560         VALUES (?, ?, ?, ?, ?, ?, ?)
561         """,
562         [
563             (
564                 task_id,
565                 row["id"],
566                 row["question"],
567                 row["answer"],
568                 row["confidence"],
569                 row["status"],
570                 json.dumps(row, ensure_ascii=False),
571             )
572             for row in results
573         ],
574     )
575
576     return {
577         "task": {
578             "id": task_id,
579             "name": task_name,
580             "createdAt": now,
581             "status": status,
582             "mode": mode,
583             "modeError": mode_error,
584             "files": saved_files,
585         },
586         "results": results,
587     }
588
589
590 @app.get("/api/tasks")
591 def list_tasks() -> dict[str, Any]:
592     init_db()
593     with connect() as db:
```

当前文件: backend/main.py

```
594         rows = db.execute("SELECT * FROM tasks ORDER BY created_at DESC").fetchall()
595         return {"tasks": [dict(row) for row in rows]}
596
597
598 @app.post("/api/export")
599 def export_excel(payload: ExportPayload) -> StreamingResponse:
600     if Workbook is None:
601         csv = "题号, 题目, 答案, 置信度, 状态\n" + "\n".join(
602             f' {row.get("id")}, "{row.get("question")}", "{row.get("answer")}", {row.get("confidence")}, {row.get("status")}'
603             for row in payload.rows
604         )
605         return StreamingResponse(
606             BytesIO(csv.encode("utf-8-sig")),
607             media_type="text/csv",
608             headers={"Content-Disposition": f' attachment; filename="{payload.taskId}.csv"'},
609         )
610
611     workbook = Workbook()
612     sheet = workbook.active
613     sheet.title = "识别结果"
614     sheet.append(["任务ID", "题号", "题目内容", "识别答案", "置信度", "处理状态", "题型"])
615     for row in payload.rows:
616         sheet.append(
617             [
618                 payload.taskId,
619                 row.get("id"),
620                 row.get("question"),
621                 row.get("answer"),
622                 row.get("confidence"),
623                 row.get("status"),
624                 row.get("type"),
625             ]
626         )
627
628     for column in sheet.columns:
629         max_length = max(len(str(cell.value or "")) for cell in column)
630         sheet.column_dimensions[column[0].column_letter].width = min(max(max_length + 4, 12), 48)
631
632     stream = BytesIO()
633     workbook.save(stream)
634     stream.seek(0)
635     return StreamingResponse(
636         stream,
637         media_type="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
638         headers={"Content-Disposition": f' attachment; filename="{payload.taskId}.xlsx"'},
639     )
640
641 // ===== File: backend/requirements.txt =====
1 fastapi
```

当前文件: backend/requirements.txt

```
2 | uvicorn[standard]
3 | python-multipart
4 | pydantic
5 | openpyxl
6 | pillow
7 | easyocr
8 | opencv-python-headless
9 | paddlepaddle==3.0.0
10 | paddleocr<3
11 | albumentations<2

    // ===== File: src/App.jsx =====
1 | import { useMemo, useRef, useState } from 'react';
2 | import {
3 |   Check,
4 |   Clock3,
5 |   Database,
6 |   Download,
7 |   FileImage,
8 |   FileSpreadsheet,
9 |   Home,
10 |  Logout,
11 |  PencilLine,
12 |  RefreshCcw,
13 |  RotateCcw,
14 |  SearchCheck,
15 |  ShieldCheck,
16 |  Trash2,
17 |  Upload,
18 |  UserRound,
19 | } from 'lucide-react';
20 |
21 | const API_BASE = import.meta.env.VITE_API_BASE || 'http://127.0.0.1:8000';
22 |
23 | const sampleRows = [
24 |   { id: 1, question: '您的性别是?', answer: '男', confidence: 98, status: '已确认', type: '单选题', options: ['男', '女'] },
25 |   { id: 2, question: '您的年龄段是?', answer: '25-35岁', confidence: 95, status: '已确认', type: '单选题', options:
26 |     ['18-24岁', '25-35岁', '36-45岁', '46岁以上'] },
27 |   { id: 3, question: '您的教育程度是?', answer: '本科', confidence: 96, status: '已确认', type: '单选题', options: ['高中及
28 | '专科', '本科', '研究生及以上'] },
29 |   { id: 4, question: '您使用本产品的频率是?', answer: '每周1-3次', confidence: 88, status: '待确认', type: '单选题', options:
30 |     ['每天多次', '每天1次', '每周1-3次', '每月4次以上', '很少使用'] },
31 |   { id: 5, question: '您最常使用的功能是? (多选)', answer: 'A、B、D', confidence: 90, status: '待确认', type: '多选题', options:
32 |     ['数据录入', 'B 报表查看', 'C 消息提醒', 'D 数据导出'] },
33 |   { id: 6, question: '您对本产品的满意度是?', answer: '满意', confidence: 93, status: '已确认', type: '单选题', options: ['
34 | '满意', '一般', '不满意'] },
35 |   { id: 7, question: '您是否会推荐本产品给他人?', answer: '是', confidence: 97, status: '已确认', type: '单选题', options:
36 |     ['否', '不确定'] },
37 |   { id: 8, question: '您对本产品有哪些建议或意见?', answer: '界面可以更简洁一些', confidence: 85, status: '待确认', type: '
38 |     [] },
```

当前文件: src/App.jsx

```
32   ];
33
34   const initialFiles = [
35     { id: 'seed-1', name: '问卷_001.jpg', size: '2.34 MB', status: '识别中 65%', progress: 65 },
36     { id: 'seed-2', name: '问卷_002.jpg', size: '1.98 MB', status: '排队中', progress: 0 },
37     { id: 'seed-3', name: '问卷_003.jpg', size: '2.21 MB', status: '排队中', progress: 0 },
38     { id: 'seed-4', name: '问卷_004.jpg', size: '1.76 MB', status: '等待中', progress: 0 },
39     { id: 'seed-5', name: '问卷_005.jpg', size: '2.02 MB', status: '等待中', progress: 0 },
40   ];
41
42   const historyRows = [
43     { id: 'TASK-20260520-0001', name: '产品体验问卷_0520', count: 5, created: '2026-05-20 10:23:45', done: '-', status:
      '处理中' },
44     { id: 'TASK-20260519-0003', name: '客户满意度调查_0519', count: 12, created: '2026-05-19 16:45:22', done: '2026-05-19
      16:48:31', status: '已完成' },
45     { id: 'TASK-20260518-0002', name: '市场调研问卷_0518', count: 8, created: '2026-05-18 09:12:11', done: '2026-05-18
      09:15:07', status: '已完成' },
46     { id: 'TASK-20260517-0001', name: '员工反馈问卷_0517', count: 15, created: '2026-05-17 14:33:05', done: '2026-05-17
      14:37:42', status: '已完成' },
47   ];
48
49   function formatBytes(bytes) {
50     if (!bytes) return '0 MB';
51     return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
52   }
53
54   function App() {
55     const fileInputRef = useRef(null);
56     const [files, setFiles] = useState(initialFiles);
57     const [rows, setRows] = useState(sampleRows);
58     const [selectedId, setSelectedId] = useState(4);
59     const [note, setNote] = useState('');
60     const [isUploading, setUploading] = useState(false);
61     const [taskId, setTaskId] = useState('TASK-20260520-0001');
62     const [lastMessage, setLastMessage] = useState('演示数据已载入, 可直接校正或导出。');
63     const [recognitionMode, setRecognitionMode] = useState('演示数据');
64
65     const selectedRow = rows.find((row) => row.id === selectedId) || rows[0];
66     const confirmedCount = rows.filter((row) => row.status === '已确认').length;
67     const taskProgress = Math.round((confirmedCount / rows.length) * 100);
68
69     const totalSize = useMemo(() => {
70       const seedTotal = files.reduce((sum, item) => {
71         const number = Number.parseFloat(String(item.size).replace(' MB', ''));
72         return sum + (Number.isNaN(number) ? 0 : number);
73       }, 0);
74       return seedTotal.toFixed(2);
75     }, [files]);
76
77     async function uploadFiles(event) {
```

当前文件: src/App.jsx

```
78     const selected = Array.from(event.target.files || []);
79     if (!selected.length) return;
80     const localFiles = selected.map((file, index) => ({
81       id: `${Date.now()}-${index}`,
82       name: file.name,
83       size: formatBytes(file.size),
84       status: index === 0 ? '上传中' : '排队中',
85       progress: index === 0 ? 35 : 0,
86     }));
87     setFiles(localFiles);
88     setUploading(true);
89     setLastMessage('正在上传图片并创建识别任务。');
90
91     try {
92       const formData = new FormData();
93       selected.forEach((file) => formData.append('files', file));
94       const response = await fetch(`${API_BASE}/api/tasks`, {
95         method: 'POST',
96         body: formData,
97       });
98       if (!response.ok) throw new Error('上传失败');
99       const data = await response.json();
100       setTaskId(data.task.id);
101       setFiles(data.task.files);
102       setRows(data.results);
103       setSelectedId(data.results[0]?.id || 1);
104       setRecognitionMode(data.task.mode === 'ocr' ? 'OCR识别模式' : '轻量识别模式');
105       setLastMessage(data.task.mode === 'ocr' ? 'OCR识别任务已完成, 结果可在表格中校正。' :
`OCR暂不可用, 已降级为轻量识别。${data.task.modeError || ''}`);
106     } catch (error) {
107       setFiles(localFiles.map((item, index) => ({
108         ...item,
109         status: index === 0 ? '识别中 65%' : item.status,
110         progress: index === 0 ? 65 : 0,
111       })));
112       setRows(sampleRows);
113       setRecognitionMode('后端未连接');
114       setLastMessage('后端服务未启动, 当前只能显示本地演示结果。请先启动 8000 后端。');
115     } finally {
116       setUploading(false);
117       event.target.value = '';
118     }
119   }
120
121   function updateAnswer(answer) {
122     setRows((current) => current.map((row) => (
123       row.id === selectedRow.id ? { ...row, answer, status: '待确认' } : row
124     )));
125   }
126
```


当前文件: src/App.jsx

```
127     function confirmSelected() {
128         setRows((current) => current.map((row) => (
129             row.id === selectedRow.id ? { ...row, status: '已确认', note } : row
130         )));
131         setLastMessage(`第 ${selectedRow.id} 题已保存校正结果。`);
132     }
133
134     function markProblem() {
135         setRows((current) => current.map((row) => (
136             row.id === selectedRow.id ? { ...row, status: '需复核' } : row
137         )));
138         setLastMessage(`第 ${selectedRow.id} 题已标记为需复核。`);
139     }
140
141     async function exportExcel() {
142         try {
143             const response = await fetch(`${API_BASE}/api/export`, {
144                 method: 'POST',
145                 headers: { 'Content-Type': 'application/json' },
146                 body: JSON.stringify({ taskId, rows }),
147             });
148             if (!response.ok) throw new Error('导出失败');
149             const blob = await response.blob();
150             const url = URL.createObjectURL(blob);
151             const link = document.createElement('a');
152             link.href = url;
153             link.download = `${taskId}_识别结果.xlsx`;
154             link.click();
155             URL.revokeObjectURL(url);
156             setLastMessage('Excel 文件已导出。');
157         } catch (error) {
158             const csv = ['题号, 题目, 答案, 置信度, 状态']
159                 .concat(rows.map((row) =>
160                     `${row.id}, "${row.question}", "${row.answer}", ${row.confidence}, ${row.status}`))
161                 .join('\n');
162             const blob = new Blob([`uFEFF${csv}`], { type: 'text/csv;charset=utf-8;' });
163             const url = URL.createObjectURL(blob);
164             const link = document.createElement('a');
165             link.href = url;
166             link.download = `${taskId}_识别结果.csv`;
167             link.click();
168             URL.revokeObjectURL(url);
169             setLastMessage('未连接后端时已导出 CSV 结果。');
170         }
171     }
172
173     return (
174         <div className="app-shell">
175             <aside className="sidebar">
```

当前文件: src/App.jsx

```
176         <ShieldCheck size={24} />
177         <span>问卷识别</span>
178     </div>
179     <nav className="nav-list" aria-label="主导航">
180         <NavItem icon={<Home size={20} />} active label="任务中心" />
181         <NavItem icon={<Upload size={20} />} label="图片上传" />
182         <NavItem icon={<SearchCheck size={20} />} label="识别结果" />
183         <NavItem icon={<PencilLine size={20} />} label="人工校正" />
184         <NavItem icon={<Database size={20} />} label="历史任务" />
185         <NavItem icon={<FileSpreadsheet size={20} />} label="数据导出" />
186     </nav>
187     <div className="storage-card">
188         <span>存储空间</span>
189         <div className="storage-track">
190             <i style={{ width: '25%' }} />
191         </div>
192         <strong>2.45 GB / 10.00 GB</strong>
193     </div>
194     <span className="version">系统版本: V1.0.0</span>
195 </aside>
196
197 <main className="workspace">
198     <header className="topbar">
199         <h1>问卷图像识别与数据导出系统 V1.0</h1>
200         <div className="top-actions">
201             <span className="user-chip"><UserRound size={18} /> 张三 <i /> 在线</span>
202             <button className="icon-button" title="退出登录"><LogOut size={18} />退出登录</button>
203         </div>
204     </header>
205
206     <section className="content-grid">
207         <div className="panel upload-panel">
208             <PanelHeader title={`上传队列 (${files.length})`} />
209             <div className="button-row">
210                 <button className="primary-button" onClick={() => fileInputRef.current?.click()}
disabled={isUploading}>
211                     <Upload size={16} /> 选择图片
212                 </button>
213                 <button className="secondary-button"><RefreshCcw size={16} /> 继续上传</button>
214                 <button className="danger-button" onClick={() => setFiles([])}><Trash2 size={16} /> 清空</button>
215                 <input ref={fileInputRef} className="hidden-input" type="file" accept=".jpg,.jpeg,.png" multiple
onChange={uploadFiles} />
216             </div>
217             <div className="file-table">
218                 <div className="file-head"><span>文件名</span><span>大小</span><span>状态</span></div>
219                 {files.map((file) => (
220                     <button
221                         className={`file-row ${file.progress > 0 ? 'selected' : ''}`}
222                         key={file.id}
223                         onClick={() => setLastMessage(`当前查看: ${file.name}`)}

```

当前文件: src/App.jsx

```
224         >
225         <span className="thumb"><FileImage size={22} /></span>
226         <span className="file-name">{file.name}</span>
227         <span>{file.size}</span>
228         <StatusText file={file} />
229     </button>
230   )))
231 </div>
232 <div className="queue-total">共 {files.length} 个文件，总大小 {totalSize} MB</div>
233 </div>
234
235 <div className="center-stack">
236   <div className="panel process-panel">
237     <PanelHeader title="处理流程" />
238     <div className={`mode-banner ${recognitionMode === '后端未连接' ? 'offline' : ''}`>
239       <strong>{recognitionMode}</strong>
240       <span>{recognitionMode === 'OCR识别模式' ? '已启用 PaddleOCR/EasyOCR 文字识别和 OpenCV 候选选项框检测。' :
recognitionMode === '轻量识别模式' ? 'OCR模型暂不可用，已读取上传图片并完成基础图像分析。' : recognitionMode === '后端未连接
127.0.0.1:8000，上传不会进入后端识别。' : '当前使用内置演示数据。'}</span>
241     </div>
242     <div className="stepper">
243       ([ '上传图片', '图像预处理', 'OCR识别', '选项识别', '结果生成' ].map((label, index) => (
244         <div className={`step ${index < 3 ? 'done' : index === 3 ? 'current' : ''}`} key={label}>
245           <span>{index + 1}</span>
246           <p>{label}</p>
247           <small>{index < 3 ? `${5 - index}/5` : index === 3 ? '2/5' : '等待中'}</small>
248         </div>
249       )))
250     </div>
251     <div className="progress-card">
252       <div><span>当前任务: </span><strong>{files[0]?.name || '暂无任务'}</strong><b>{taskProgress}%</b></div>
253       <div className="progress-track"><i style={{ width: `${Math.max(65, taskProgress)}%` }} /></div>
254       <p>正在进行: 选项检测与识别 <span>预计剩余时间: 00:00:18</span></p>
255     </div>
256   </div>
257
258   <div className="panel result-panel">
259     <div className="panel-titlebar">
260       <h2>识别结果预览 ( {files[0]?.name || '问卷_001.jpg'} ) </h2>
261       <div className="confidence">置信度: {Math.round(rows.reduce((sum, row) => sum + row.confidence, 0) /
rows.length)}%</div>
262       <button className="secondary-button compact"><RefreshCcw size={15} /> 刷新</button>
263       <select aria-label="显示题目范围">
264         <option>显示: 全部题目</option>
265         <option>仅待确认</option>
266         <option>仅需复核</option>
267       </select>
268     </div>
269     <table className="result-table">
270       <thead>
```

当前文件: src/App.jsx

```
271         <tr>
272             <th>题号</th>
273             <th>题目内容</th>
274             <th>识别结果</th>
275             <th>置信度</th>
276             <th>状态</th>
277             <th>操作</th>
278         </tr>
279     </thead>
280     <tbody>
281         {rows.map((row) => (
282             <tr key={row.id} className={row.id === selectedRow.id ? 'active-row' : ''} onClick={() =>
setSelectedId(row.id)}>
283                 <td>{row.id}</td>
284                 <td>{row.question}</td>
285                 <td>{row.answer}</td>
286                 <td>{row.confidence}%</td>
287                 <td><StatusBadge status={row.status} /></td>
288                 <td><button className="link-button" onClick={() => setSelectedId(row.id)}>校正</button></td>
289             </tr>
290         ))}
291     </tbody>
292 </table>
293 <div className="table-footer">
294     <span>共 {rows.length} 题</span>
295     <div className="pager"><button
disabled></button><strong>1</strong><button></button><select><option>20 条/页</option></select></div>
296 </div>
297 </div>
298
299 <div className="panel batch-panel">
300     <strong>批量操作</strong>
301     <span>已选择 {rows.filter((row) => row.status !== '已确认').length} 项</span>
302     <button className="secondary-button">批量确认</button>
303     <button className="secondary-button teal">批量校正</button>
304     <button className="danger-button">标记有问题</button>
305     <button className="secondary-button"><RotateCcw size={15} /> 重新识别</button>
306 </div>
307 </div>
308
309 <aside className="right-stack">
310     <div className="panel task-info">
311         <PanelHeader title="任务信息" />
312         <InfoRow label="任务ID: " value={taskId} />
313         <InfoRow label="创建时间: " value="2026-06-24 10:23:45" />
314         <InfoRow label="图片数量: " value={` ${files.length} 张`} />
315         <InfoRow label="完成数量: " value={` ${confirmedCount} 题`} />
316         <InfoRow label="任务状态: " value="处理中" tone="teal" />
317     </div>
318
```

当前文件: src/App.jsx

```
319     <div className="panel inspector">
320       <div className="tabs"><button className="active">题目详情</button><button>原图预览</button></div>
321       <h3>题目 {selectedRow.id}: {selectedRow.question}</h3>
322       <p>题型: {selectedRow.type}</p>
323       <label className="field-label">识别结果</label>
324       <input value={selectedRow.answer} onChange={(event) => updateAnswer(event.target.value)} />
325       {selectedRow.options.length > 0 && (
326         <>
327           <label className="field-label">选项列表</label>
328           <div className="option-list">
329             {selectedRow.options.map((option) => (
330               <label key={option} className={selectedRow.answer.includes(option) ? 'checked' : ''}>
331                 <input
332                   type={selectedRow.type === '多选题' ? 'checkbox' : 'radio'}
333                   name="answer"
334                   checked={selectedRow.answer.includes(option)}
335                   onChange={() => updateAnswer(option)}
336                 />
337                 <span>{option}</span>
338                 <em>{selectedRow.answer.includes(option) ? `选中 (${selectedRow.confidence}%)` :
'未选中'}</em>
339               </label>
340             ))}
341           </div>
342         </>
343       )}
344       <label className="field-label">校正操作</label>
345       <div className="correction-row">
346         <button className="primary-button" onClick={confirmSelected}><Check size={16} /> 确认正确</button>
347         <button className="warning-button" onClick={() => setLastMessage('已进入修改选择模式。')}><PencilLine
size={16} /> 修改选择</button>
348         <button className="danger-button" onClick={() => updateAnswer('')}><Trash2 size={16} />
清除选择</button>
349       </div>
350       <textarea placeholder="请输入备注信息 (可选)" value={note} onChange={(event) => setNote(event.target.value)}
/>
351       <button className="mark-button" onClick={markProblem}>标记为需复核</button>
352     </div>
353   </aside>
354 </section>
355
356   <section className="panel history-panel">
357     <div className="panel-titlebar">
358       <h2>任务历史</h2>
359       <div className="history-actions">
360         <button className="secondary-button"><Clock3 size={15} /> 查看更多历史</button>
361         <button className="primary-button" onClick={exportExcel}><Download size={16} />
导出全部结果 (Excel) </button>
362       </div>
363     </div>
```

当前文件: src/App.jsx

```
364     <table className="history-table">
365       <thead>
366         <tr><th>任务ID</th><th>任务名称</th><th>图片数量</th><th>创建时间</th><th>完成时间</th><th>状态</th><th>操作</th></tr>
367       </thead>
368       <tbody>
369         {historyRows.map((row) => (
370           <tr key={row.id}>
371             <td>{row.id}</td><td>{row.name}</td><td>{row.count}</td><td>{row.created}</td><td>{row.done}</td>
372             <td><StatusBadge status={row.status} /></td>
373             <td><button className="link-button">查看</button><button className="link-
button">导出</button><button className="link-button red">删除</button></td>
374           </tr>
375         ))}
376       </tbody>
377     </table>
378   </section>
379   <div className="toast" role="status">{lastMessage}</div>
380 </main>
381 </div>
382 );
383 }
384
385 function NavItem({ icon, label, active }) {
386   return <button className={`nav-item ${active ? 'active' : ''}`}>{icon}<span>{label}</span></button>;
387 }
388
389 function PanelHeader({ title }) {
390   return <div className="panel-heading"><h2>{title}</h2></div>;
391 }
392
393 function InfoRow({ label, value, tone }) {
394   return <div className="info-row"><span>{label}</span><strong className={tone}>{value}</strong></div>;
395 }
396
397 function StatusText({ file }) {
398   if (file.progress > 0) return <span className="file-status teal"><span className="spinner"
/>{file.status}</span>;
399   if (file.status.includes('排队')) return <span className="file-status blue"><Clock3 size={16}
/>{file.status}</span>;
400   return <span className="file-status muted"><Clock3 size={16} />{file.status}</span>;
401 }
402
403 function StatusBadge({ status }) {
404   const className = status === '已确认' || status === '已完成' ? 'ok' : status === '需复核' ? 'danger' : status === '处理中'
? 'processing' : 'pending';
405   return <span className={`status-badge ${className}`}>{status}</span>;
406 }
407
408 export default App;
```

当前文件: src/App.jsx

```
// ===== File: src/main.jsx =====
1 import React from 'react';
2 import { createRoot } from 'react-dom/client';
3 import App from './App.jsx';
4 import './styles.css';
5
6 createRoot(document.getElementById('root')).render(
7   <React.StrictMode>
8     <App />
9   </React.StrictMode>,
10 );

// ===== File: src/styles.css =====
1 :root {
2   color-scheme: light;
3   font-family: "Microsoft YaHei", "PingFang SC", "Segoe UI", Arial, sans-serif;
4   background: #f5f7fb;
5   color: #172033;
6   --bg: #f5f7fb;
7   --surface: #ffffff;
8   --line: #dbe3ee;
9   --line-strong: #c6d1df;
10  --muted: #687386;
11  --text: #172033;
12  --soft: #eef4f8;
13  --blue: #2867d6;
14  --blue-soft: #e8f1ff;
15  --teal: #0f9f9a;
16  --teal-soft: #e1f6f4;
17  --green: #1a9b63;
18  --green-soft: #e5f8ee;
19  --amber: #c47512;
20  --amber-soft: #fff3da;
21  --red: #cf3d3d;
22  --red-soft: #ffebeb;
23  --shadow: 0 14px 32px rgba(35, 47, 70, 0.08);
24 }
25
26 * {
27   box-sizing: border-box;
28 }
29
30 body {
31   margin: 0;
32   min-width: 1180px;
33   background: var(--bg);
34 }
35
36 button,
```

当前文件: src/styles.css

```
37 | input,
38 | select,
39 | textarea {
40 |   font: inherit;
41 | }
42 |
43 | button {
44 |   cursor: pointer;
45 | }
46 |
47 | .app-shell {
48 |   min-height: 100vh;
49 |   display: grid;
50 |   grid-template-columns: 232px minmax(0, 1fr);
51 | }
52 |
53 | .sidebar {
54 |   background: #172033;
55 |   color: #eaf0f7;
56 |   padding: 22px 16px;
57 |   display: flex;
58 |   flex-direction: column;
59 |   gap: 18px;
60 | }
61 |
62 | .brand-mark {
63 |   display: flex;
64 |   align-items: center;
65 |   gap: 10px;
66 |   font-weight: 800;
67 |   font-size: 20px;
68 |   height: 40px;
69 | }
70 |
71 | .brand-mark svg {
72 |   color: #59d6d0;
73 | }
74 |
75 | .nav-list {
76 |   display: grid;
77 |   gap: 6px;
78 | }
79 |
80 | .nav-item {
81 |   border: 0;
82 |   background: transparent;
83 |   color: #aeb9c8;
84 |   height: 42px;
85 |   display: flex;
86 |   align-items: center;
```


当前文件: [src/styles.css](#)

```
87     gap: 10px;
88     padding: 0 12px;
89     border-radius: 6px;
90     text-align: left;
91     transition: background 160ms ease, color 160ms ease;
92 }
93
94 .nav-item:hover,
95 .nav-item.active {
96     background: rgba(77, 214, 208, 0.15);
97     color: #ffffff;
98 }
99
100 .storage-card {
101     margin-top: auto;
102     padding: 14px;
103     background: rgba(255, 255, 255, 0.08);
104     border: 1px solid rgba(255, 255, 255, 0.1);
105     border-radius: 8px;
106     display: grid;
107     gap: 8px;
108     color: #cfd8e5;
109     font-size: 13px;
110 }
111
112 .storage-track,
113 .progress-track {
114     height: 8px;
115     background: #e4eaf3;
116     border-radius: 99px;
117     overflow: hidden;
118 }
119
120 .storage-track {
121     background: rgba(255, 255, 255, 0.16);
122 }
123
124 .storage-track i,
125 .progress-track i {
126     display: block;
127     height: 100%;
128     background: linear-gradient(90deg, var(--teal), #2a77dd);
129 }
130
131 .version {
132     color: #8290a3;
133     font-size: 12px;
134 }
135
136 .workspace {
```

当前文件: src/styles.css

```
137     min-width: 0;
138     padding: 18px 22px 20px;
139     display: grid;
140     gap: 16px;
141 }
142
143 .topbar {
144     height: 54px;
145     display: flex;
146     align-items: center;
147     justify-content: space-between;
148 }
149
150 .topbar h1 {
151     margin: 0;
152     font-size: 22px;
153     line-height: 1.2;
154     letter-spacing: 0;
155 }
156
157 .top-actions {
158     display: flex;
159     align-items: center;
160     gap: 10px;
161 }
162
163 .user-chip,
164 .icon-button,
165 .secondary-button,
166 .danger-button,
167 .primary-button,
168 .warning-button,
169 .mark-button {
170     height: 34px;
171     display: inline-flex;
172     align-items: center;
173     justify-content: center;
174     gap: 7px;
175     border-radius: 6px;
176     border: 1px solid transparent;
177     white-space: nowrap;
178     font-size: 13px;
179     font-weight: 600;
180 }
181
182 .user-chip {
183     background: var(--surface);
184     border: 1px solid var(--line);
185     padding: 0 12px;
186     color: var(--muted);
```

当前文件: src/styles.css

```
187 | }
188 |
189 | .user-chip i {
190 |   width: 8px;
191 |   height: 8px;
192 |   border-radius: 50%;
193 |   background: var(--green);
194 | }
195 |
196 | .icon-button,
197 | .secondary-button {
198 |   background: var(--surface);
199 |   color: #344054;
200 |   border-color: var(--line);
201 |   padding: 0 12px;
202 | }
203 |
204 | .primary-button {
205 |   background: var(--blue);
206 |   color: white;
207 |   padding: 0 14px;
208 | }
209 |
210 | .danger-button {
211 |   background: var(--red-soft);
212 |   color: var(--red);
213 |   border-color: #ffd2d2;
214 |   padding: 0 12px;
215 | }
216 |
217 | .warning-button {
218 |   background: var(--amber-soft);
219 |   color: var(--amber);
220 |   border-color: #ffe0a8;
221 |   padding: 0 12px;
222 | }
223 |
224 | .secondary-button.teal {
225 |   color: #087d79;
226 |   background: var(--teal-soft);
227 |   border-color: #bdebe7;
228 | }
229 |
230 | .compact {
231 |   height: 30px;
232 | }
233 |
234 | .content-grid {
235 |   display: grid;
236 |   grid-template-columns: 334px minmax(520px, 1fr) 330px;
```

当前文件: [src/styles.css](#)

```
237     gap: 16px;
238     align-items: start;
239   }
240
241   .center-stack,
242   .right-stack {
243     display: grid;
244     gap: 16px;
245   }
246
247   .panel {
248     background: var(--surface);
249     border: 1px solid var(--line);
250     border-radius: 8px;
251     box-shadow: var(--shadow);
252   }
253
254   .panel-heading,
255   .panel-titlebar {
256     min-height: 48px;
257     padding: 0 16px;
258     display: flex;
259     align-items: center;
260     justify-content: space-between;
261     border-bottom: 1px solid var(--line);
262   }
263
264   .panel-heading h2,
265   .panel-titlebar h2 {
266     margin: 0;
267     font-size: 15px;
268     line-height: 1.2;
269   }
270
271   .button-row {
272     padding: 14px 14px 8px;
273     display: flex;
274     gap: 8px;
275     flex-wrap: wrap;
276   }
277
278   .hidden-input {
279     display: none;
280   }
281
282   .file-table {
283     padding: 4px 14px 0;
284     display: grid;
285   }
286
```

当前文件: src/styles.css

```
287 | .file-head,
288 | .file-row {
289 |     display: grid;
290 |     grid-template-columns: 32px minmax(82px, 1fr) 58px 72px;
291 |     align-items: center;
292 |     column-gap: 7px;
293 | }
294 |
295 | .file-head {
296 |     grid-template-columns: minmax(114px, 1fr) 58px 72px;
297 |     padding: 0 6px 8px 39px;
298 |     color: var(--muted);
299 |     font-size: 12px;
300 | }
301 |
302 | .file-row {
303 |     min-height: 54px;
304 |     border: 1px solid transparent;
305 |     background: transparent;
306 |     border-radius: 6px;
307 |     width: 100%;
308 |     color: var(--text);
309 |     text-align: left;
310 |     padding: 0 6px;
311 |     font-size: 13px;
312 |     overflow: hidden;
313 | }
314 |
315 | .file-row:hover,
316 | .file-row.selected {
317 |     background: #f7fbff;
318 |     border-color: #cde0f8;
319 | }
320 |
321 | .thumb {
322 |     width: 30px;
323 |     height: 34px;
324 |     border-radius: 5px;
325 |     background: var(--blue-soft);
326 |     color: var(--blue);
327 |     display: grid;
328 |     place-items: center;
329 | }
330 |
331 | .file-name {
332 |     overflow: hidden;
333 |     text-overflow: ellipsis;
334 |     white-space: nowrap;
335 |     font-weight: 600;
336 | }
```

当前文件: src/styles.css

```
337
338 .file-status {
339     display: flex;
340     align-items: center;
341     gap: 4px;
342     font-size: 11px;
343     font-weight: 700;
344     min-width: 0;
345     white-space: nowrap;
346 }
347
348 .file-status.teal {
349     color: var(--teal);
350 }
351
352 .file-status.blue {
353     color: var(--blue);
354 }
355
356 .file-status.muted {
357     color: var(--muted);
358 }
359
360 .spinner {
361     width: 12px;
362     height: 12px;
363     border-radius: 50%;
364     border: 2px solid #b7e9e6;
365     border-top-color: var(--teal);
366     animation: spin 0.9s linear infinite;
367 }
368
369 .queue-total {
370     margin: 12px 14px 14px;
371     padding: 10px 12px;
372     background: #f6f8fb;
373     border-radius: 6px;
374     color: var(--muted);
375     font-size: 13px;
376 }
377
378 .process-panel {
379     padding-bottom: 14px;
380 }
381
382 .mode-banner {
383     margin: 12px 16px 0;
384     min-height: 42px;
385     padding: 9px 12px;
386     display: grid;
```

当前文件: [src/styles.css](#)

```
387     gap: 3px;
388     border: 1px solid #bdebe7;
389     border-radius: 6px;
390     background: var(--teal-soft);
391     color: #075f5c;
392     font-size: 12px;
393 }
394
395 .mode-banner strong {
396     font-size: 13px;
397 }
398
399 .mode-banner.offline {
400     border-color: #ffd2d2;
401     background: var(--red-soft);
402     color: var(--red);
403 }
404
405 .stepper {
406     display: grid;
407     grid-template-columns: repeat(5, 1fr);
408     gap: 6px;
409     padding: 14px 16px;
410 }
411
412 .step {
413     position: relative;
414     display: grid;
415     gap: 5px;
416     align-items: center;
417     justify-items: center;
418     color: var(--muted);
419 }
420
421 .step span {
422     width: 28px;
423     height: 28px;
424     border-radius: 50%;
425     display: grid;
426     place-items: center;
427     background: #e8edf5;
428     color: #536175;
429     font-size: 13px;
430     font-weight: 800;
431 }
432
433 .step.done span {
434     background: var(--green);
435     color: white;
436 }
```

当前文件: src/styles.css

```
437
438 .step.current span {
439     background: var(--teal);
440     color: white;
441 }
442
443 .step p,
444 .step small {
445     margin: 0;
446     font-size: 12px;
447     text-align: center;
448 }
449
450 .step p {
451     color: var(--text);
452     font-weight: 700;
453 }
454
455 .progress-card {
456     margin: 0 16px;
457     padding: 13px;
458     background: #f7fbff;
459     border: 1px solid #d7e7fb;
460     border-radius: 8px;
461     display: grid;
462     gap: 9px;
463 }
464
465 .progress-card div:first-child {
466     display: flex;
467     align-items: center;
468     gap: 7px;
469     font-size: 13px;
470 }
471
472 .progress-card b {
473     margin-left: auto;
474     color: var(--teal);
475 }
476
477 .progress-card p {
478     margin: 0;
479     display: flex;
480     justify-content: space-between;
481     color: var(--muted);
482     font-size: 12px;
483 }
484
485 .panel-titlebar {
486     gap: 10px;
```


当前文件: src/styles.css

```
487 | }
488 |
489 | .panel-titlebar select {
490 |     height: 30px;
491 |     border: 1px solid var(--line);
492 |     border-radius: 6px;
493 |     background: white;
494 |     color: #3b4656;
495 |     font-size: 13px;
496 |     padding: 0 8px;
497 | }
498 |
499 | .confidence {
500 |     margin-left: auto;
501 |     color: var(--teal);
502 |     font-size: 13px;
503 |     font-weight: 800;
504 | }
505 |
506 | table {
507 |     width: 100%;
508 |     border-collapse: collapse;
509 | }
510 |
511 | .result-table th,
512 | .result-table td,
513 | .history-table th,
514 | .history-table td {
515 |     border-bottom: 1px solid var(--line);
516 |     padding: 11px 12px;
517 |     font-size: 13px;
518 |     text-align: left;
519 | }
520 |
521 | .result-table th,
522 | .history-table th {
523 |     background: #f7f9fc;
524 |     color: #566276;
525 |     font-size: 12px;
526 |     font-weight: 800;
527 | }
528 |
529 | .result-table tbody tr,
530 | .history-table tbody tr {
531 |     transition: background 120ms ease;
532 | }
533 |
534 | .result-table tbody tr:hover,
535 | .result-table tbody tr.active-row {
536 |     background: #f2f8ff;
```

当前文件: src/styles.css

```
537 | }
538 |
539 | .status-badge {
540 |     display: inline-flex;
541 |     align-items: center;
542 |     min-width: 58px;
543 |     height: 24px;
544 |     justify-content: center;
545 |     border-radius: 99px;
546 |     font-size: 12px;
547 |     font-weight: 800;
548 | }
549 |
550 | .status-badge.ok {
551 |     color: var(--green);
552 |     background: var(--green-soft);
553 | }
554 |
555 | .status-badge.pending {
556 |     color: var(--amber);
557 |     background: var(--amber-soft);
558 | }
559 |
560 | .status-badge.danger {
561 |     color: var(--red);
562 |     background: var(--red-soft);
563 | }
564 |
565 | .status-badge.processing {
566 |     color: var(--blue);
567 |     background: var(--blue-soft);
568 | }
569 |
570 | .link-button {
571 |     border: 0;
572 |     background: transparent;
573 |     color: var(--blue);
574 |     padding: 3px 5px;
575 |     font-weight: 700;
576 | }
577 |
578 | .link-button.red {
579 |     color: var(--red);
580 | }
581 |
582 | .table-footer {
583 |     padding: 12px 16px;
584 |     display: flex;
585 |     align-items: center;
586 |     justify-content: space-between;
```

当前文件: [src/styles.css](#)

```
587     color: var(--muted);
588     font-size: 13px;
589 }
590
591 .pager {
592     display: flex;
593     align-items: center;
594     gap: 8px;
595 }
596
597 .pager button {
598     width: 28px;
599     height: 28px;
600     border: 1px solid var(--line);
601     background: white;
602     border-radius: 5px;
603 }
604
605 .pager strong {
606     width: 28px;
607     height: 28px;
608     display: grid;
609     place-items: center;
610     color: white;
611     background: var(--blue);
612     border-radius: 5px;
613 }
614
615 .pager select {
616     height: 28px;
617     border: 1px solid var(--line);
618     border-radius: 5px;
619 }
620
621 .batch-panel {
622     min-height: 54px;
623     padding: 10px 14px;
624     display: flex;
625     align-items: center;
626     gap: 10px;
627 }
628
629 .batch-panel strong {
630     font-size: 14px;
631 }
632
633 .batch-panel span {
634     color: var(--muted);
635     font-size: 13px;
636     margin-right: auto;
```

当前文件: src/styles.css

```
637 | }
638 |
639 | .task-info {
640 |     padding-bottom: 12px;
641 | }
642 |
643 | .info-row {
644 |     padding: 10px 16px 0;
645 |     display: flex;
646 |     justify-content: space-between;
647 |     gap: 12px;
648 |     font-size: 13px;
649 | }
650 |
651 | .info-row span {
652 |     color: var(--muted);
653 | }
654 |
655 | .teal {
656 |     color: var(--teal);
657 | }
658 |
659 | .inspector {
660 |     padding-bottom: 14px;
661 | }
662 |
663 | .tabs {
664 |     display: grid;
665 |     grid-template-columns: 1fr 1fr;
666 |     border-bottom: 1px solid var(--line);
667 | }
668 |
669 | .tabs button {
670 |     height: 42px;
671 |     border: 0;
672 |     background: white;
673 |     color: var(--muted);
674 |     font-weight: 800;
675 |     border-bottom: 2px solid transparent;
676 | }
677 |
678 | .tabs button.active {
679 |     color: var(--blue);
680 |     border-bottom-color: var(--blue);
681 | }
682 |
683 | .inspector h3 {
684 |     margin: 14px 16px 6px;
685 |     font-size: 15px;
686 |     line-height: 1.45;
```

当前文件: src/styles.css

```
687 | }
688 |
689 | .inspector p {
690 |     margin: 0 16px 14px;
691 |     color: var(--muted);
692 |     font-size: 13px;
693 | }
694 |
695 | .field-label {
696 |     margin: 12px 16px 7px;
697 |     display: block;
698 |     color: #3d4858;
699 |     font-size: 13px;
700 |     font-weight: 800;
701 | }
702 |
703 | .inspector input[type="text"],
704 | .inspector input:not([type]),
705 | .inspector textarea,
706 | .inspector > input {
707 |     width: calc(100% - 32px);
708 |     margin: 0 16px;
709 |     border: 1px solid var(--line-strong);
710 |     border-radius: 6px;
711 |     min-height: 36px;
712 |     padding: 8px 10px;
713 |     color: var(--text);
714 |     outline: none;
715 | }
716 |
717 | .inspector input:focus,
718 | .inspector textarea:focus {
719 |     border-color: var(--blue);
720 |     box-shadow: 0 0 0 3px rgba(40, 103, 214, 0.12);
721 | }
722 |
723 | .option-list {
724 |     margin: 0 16px;
725 |     display: grid;
726 |     gap: 8px;
727 | }
728 |
729 | .option-list label {
730 |     min-height: 42px;
731 |     padding: 9px 10px;
732 |     display: grid;
733 |     grid-template-columns: 18px minmax(0, 1fr) auto;
734 |     gap: 8px;
735 |     align-items: center;
736 |     border: 1px solid var(--line);
```

当前文件: src/styles.css

```
737     border-radius: 6px;
738     background: white;
739 }
740
741 .option-list label.checked {
742     background: var(--teal-soft);
743     border-color: #a3e3df;
744 }
745
746 .option-list em {
747     color: var(--muted);
748     font-size: 12px;
749     font-style: normal;
750 }
751
752 .correction-row {
753     display: flex;
754     gap: 7px;
755     margin: 0 16px 10px;
756     flex-wrap: wrap;
757 }
758
759 .inspector textarea {
760     min-height: 74px;
761     resize: vertical;
762 }
763
764 .mark-button {
765     margin: 12px 16px 0;
766     width: calc(100% - 32px);
767     background: #f7f9fc;
768     color: #344054;
769     border-color: var(--line);
770 }
771
772 .history-panel {
773     overflow: hidden;
774 }
775
776 .history-actions {
777     display: flex;
778     gap: 8px;
779 }
780
781 .toast {
782     position: fixed;
783     right: 22px;
784     bottom: 18px;
785     min-width: 280px;
786     min-height: 42px;
```

当前文件: src/styles.css

```
787     padding: 11px 14px;
788     border-radius: 8px;
789     background: #172033;
790     color: white;
791     box-shadow: 0 18px 36px rgba(23, 32, 51, 0.25);
792     font-size: 13px;
793 }
794
795 @keyframes spin {
796     to {
797         transform: rotate(360deg);
798     }
799 }
800
801 @media (max-width: 900px) {
802     body {
803         min-width: 0;
804     }
805
806     .app-shell {
807         grid-template-columns: 1fr;
808     }
809
810     .sidebar {
811         display: none;
812     }
813
814     .workspace {
815         padding: 12px;
816     }
817
818     .topbar {
819         height: auto;
820         align-items: flex-start;
821         gap: 12px;
822         flex-direction: column;
823     }
824
825     .content-grid {
826         grid-template-columns: 1fr;
827     }
828
829     .stepper {
830         grid-template-columns: 1fr;
831     }
832
833     .panel-titlebar {
834         flex-wrap: wrap;
835         padding: 10px 12px;
836     }
```

当前文件: src/styles.css

```
837
838 .result-panel,
839 .history-panel {
840     overflow-x: auto;
841 }
842
843 .result-table,
844 .history-table {
845     min-width: 720px;
846 }
847
848 .batch-panel {
849     flex-wrap: wrap;
850 }
851 }

// ===== File: package.json =====
1 {
2     "scripts": {
3         "dev": "vite --host 127.0.0.1",
4         "build": "vite build",
5         "preview": "vite preview --host 127.0.0.1"
6     },
7     "dependencies": {
8         "@vitejs/plugin-react": "latest",
9         "lucide-react": "latest",
10        "react": "latest",
11        "react-dom": "latest",
12        "vite": "latest"
13    },
14    "devDependencies": {
15        "playwright": "^1.61.1"
16    }
17 }

// ===== File: index.html =====
1 <!doctype html>
2 <html lang="zh-CN">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>问卷图像识别与数据导出系统 V1.0</title>
7   </head>
8   <body>
9     <div id="root"></div>
10    <script type="module" src="/src/main.jsx"></script>
11  </body>
12 </html>
```